

Chapter 4. Using LangGraph to Add Memory to Your Chatbot

In [Chapter 3](#), you learned how to provide your AI chatbot application with up-to-date and relevant context. This enables your chatbot to generate accurate responses based on the user's input. But that's not enough to build a production-ready application. How can you enable your application to actually “chat” back and forth with the user, while remembering prior conversations and relevant context?

Large language models are *stateless*, which means that each time the model is prompted to generate a new response it has no memory of the prior prompt or model response. In order to provide this historical information to the model, we need a robust memory system that will keep track of previous conversations and context. This historical information can then be included in the final prompt sent to the LLM, thus giving it “memory.” [Figure 4-1](#) illustrates this.

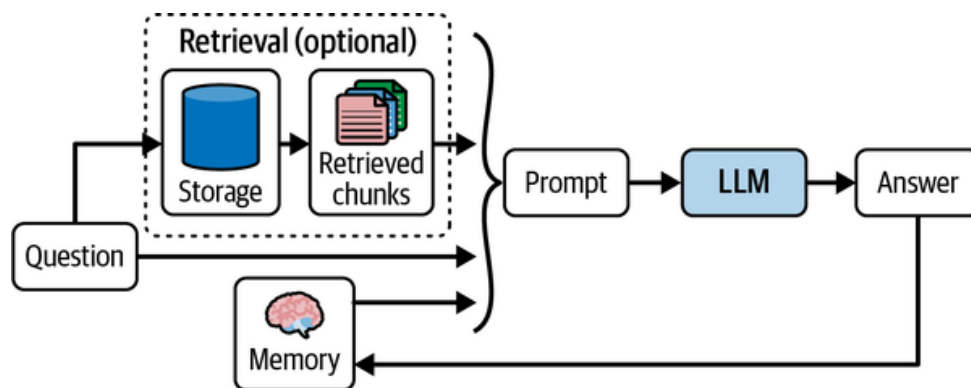


Figure 4-1. Memory and retrieval used to generate context-aware answers from an LLM

In this chapter, you'll learn how to build this essential memory system using LangChain's built-in modules to make this development process easier.

Building a Chatbot Memory System

There are two core design decisions behind any robust memory system:

- How state is stored
- How state is queried

A simple way to build a chatbot memory system that incorporates effective solutions to these design decisions is to store and reuse the history of all chat interactions between the user and the model. The state of this memory system can be:

- Stored as a list of messages (refer to [Chapter 1](#) to learn more about messages)
- Updated by appending recent messages after each turn
- Appended into the prompt by inserting the messages into the prompt

[Figure 4-2](#) illustrates this simple memory system.

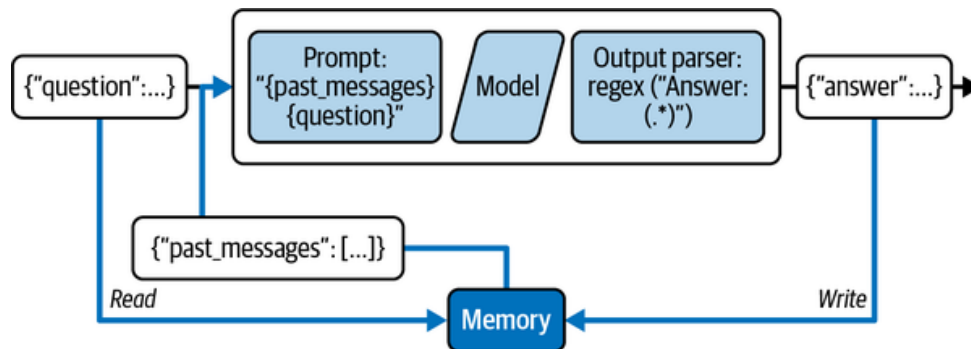


Figure 4-2. A simple memory system utilizing chat history in prompts to generate model answers

Here's a code example that illustrates a simple version of this memory system using LangChain:

Python

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a helpful assistant. Answer all questions to the
    of your ability."""),
    ("placeholder", "{messages}"),
])

model = ChatOpenAI()

chain = prompt | model

chain.invoke({
    "messages": [
        ("human", """"Translate this sentence from English to French: I love
        programming."""),
        ("ai", "J'adore programmer."),
        ("human", "What did you just say?"),
    ],
})
```

```
import {ChatPromptTemplate} from '@langchain/core/prompts'
import {ChatOpenAI} from '@langchain/openai'

const prompt = ChatPromptTemplate.fromMessages([
  ["system", `You are a helpful assistant. Answer all questions to the best
    of your ability.`],
  ["placeholder", "{messages}"],
])

const model = new ChatOpenAI()

const chain = prompt.pipe(model)

await chain.invoke({
  "messages": [
    ["human", `Translate this sentence from English to French: I love
      programming.`],
    ["ai", "J'adore programmer."],
    ["human", "What did you just say?"],
  ],
})
```

The output:

```
I said, "J'adore programmer," which means "I love programming" in French.
```

Note how the incorporation of the previous conversation in the chain enabled the model to answer the follow-up question in a context-aware manner.

While this is simple and it works, when taking your application to production, you'll face some more challenges related to managing memory at scale, such as:

- You'll need to update the memory after every interaction, atomically (i.e., don't record only the question or only the answer in the case of failure).
- You'll want to store these memories in durable storage, such as a relational database.
- You'll want to control how many and which messages are stored for later, and how many of these are used for new interactions.
- You'll want to inspect and modify this state (for now, just a list of messages) outside a call to an LLM.

We'll now introduce some better tooling, which will help with this and all later chapters.

Introducing LangGraph

For the remainder of this chapter and the following chapters, we'll start to make use of [LangGraph](#), an open source library authored by LangChain. LangGraph was designed to enable developers to implement multiactor, multistep, stateful cognitive architectures, called *graphs*. That's a lot of words packed into a short sentence; let's take them one at a time. [Figure 4-3](#) illustrates the multiactor aspect.

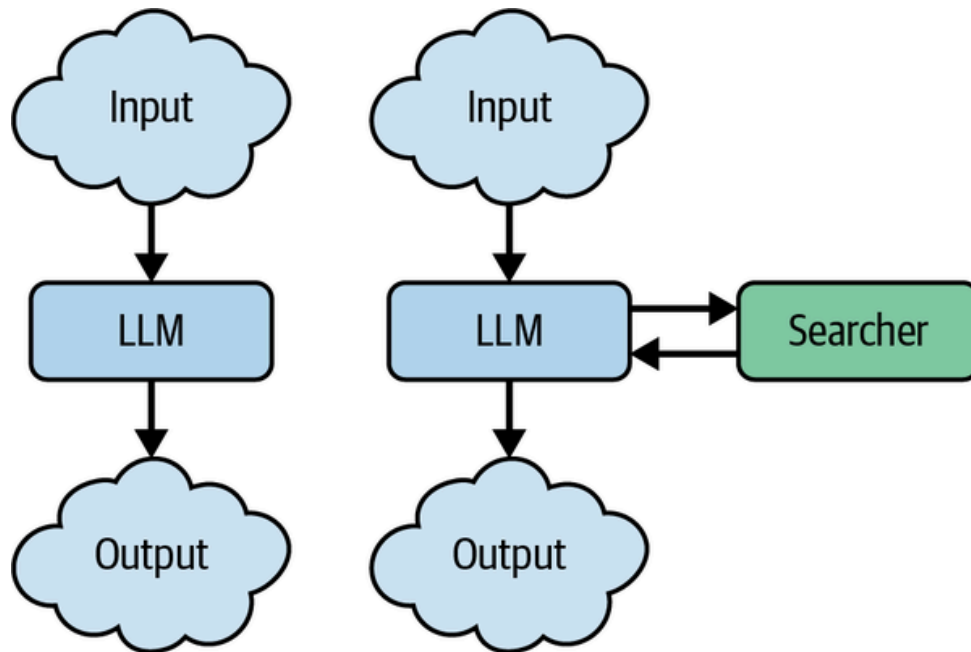


Figure 4-3. From single-actor applications to multiactor applications

A team of specialists can build something together that none of them could build alone. The same is true of LLM applications: an LLM prompt (great for answer generation and task planning and many more things) is much more powerful when paired up with a search engine (best at finding current facts), or even when paired with different LLM prompts. We have seen developers build some amazing applications, like [Perplexity](#) or [Arc Search](#), when they combine those two building blocks (and others) in novel ways.

And just as a human team needs more coordination than one person working by themselves, an application with multiple actors needs a coordination layer to do these things:

- Define the actors involved (the nodes in a graph) and how they hand off work to each other (the edges in that graph).
- Schedule execution of each actor at the appropriate time—in parallel if needed—with deterministic results.

[Figure 4-4](#) illustrates the multistep dimension.

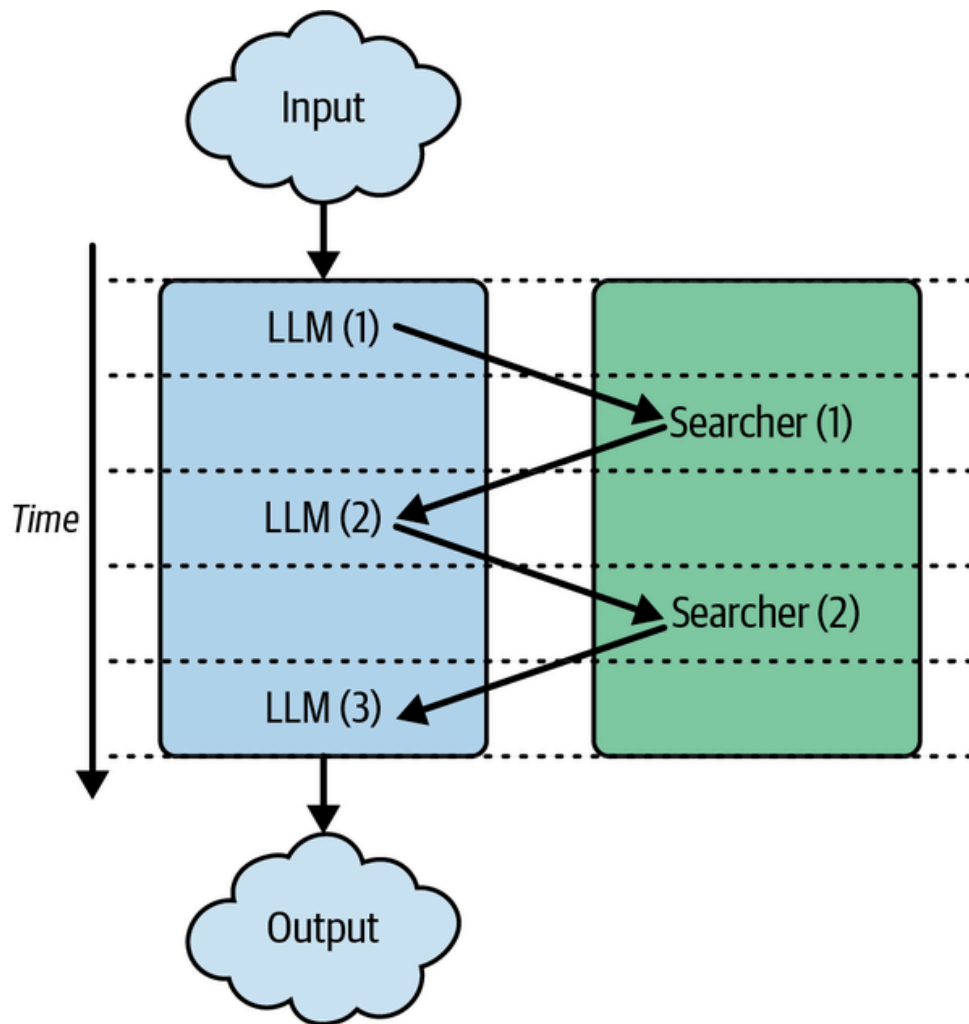


Figure 4-4. From multiactor to multistep applications

As each actor hands off work to another (for example, an LLM prompt asking a search tool for the results of a given search query), we need to make sense of the back-and-forth between multiple actors. We need to know what order it happens in, how many times each actor is called, and so on. To do this, we can model the interaction between the actors as happening across multiple discrete steps in time. When one actor hands off work to another actor, it results in the scheduling of the next step of the computation, and so on, until no more actors hand off work to others, and the final result is reached.

[Figure 4-5](#) illustrates the stateful aspect.

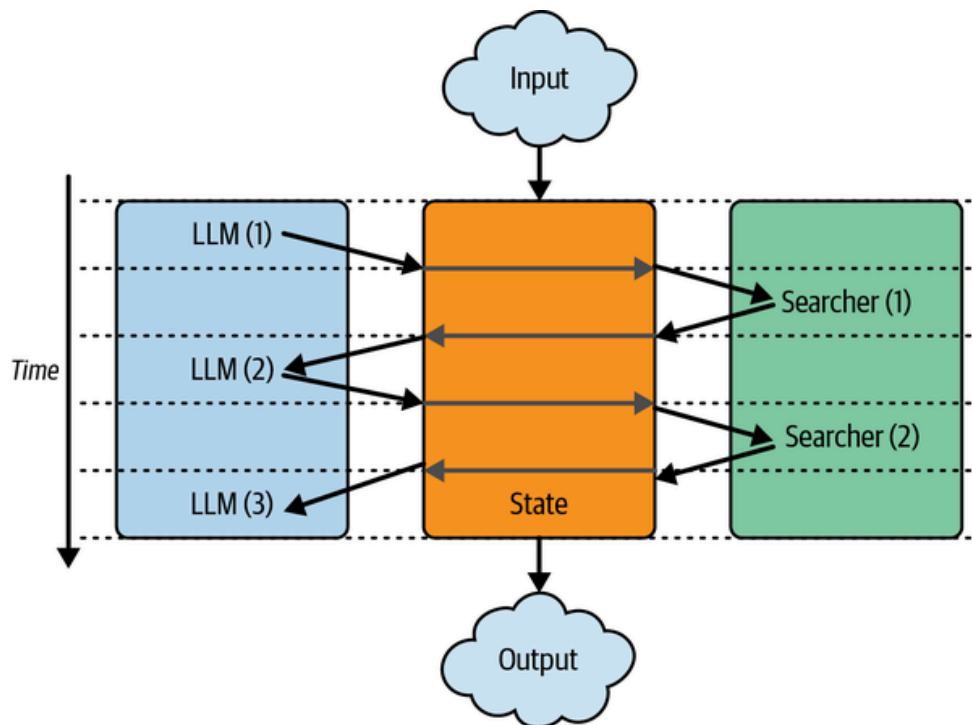


Figure 4-5. From multistep to stateful applications

Communication across steps requires tracking some state—otherwise, when you call the LLM actor the second time, you’d get the same result as the first time. It is very helpful to pull this state out of each of the actors and have all actors collaborate on updating a single central state. With a single central state, we can:

- Snapshot and store the central state during or after each computation.
- Pause and resume execution, which makes it easy to recover from errors.
- Implement human-in-the-loop controls (more on this in [Chapter 8](#)).

Each *graph* is then made up of the following:

State

The data received from outside the application, modified and produced by the application while it’s running.

Nodes

Each step to be taken. Nodes are simply Python/JS functions, which receive the current state as input and can return an update to that state (that is, they can add to it and modify or remove existing data).

Edges

The connections between nodes. Edges determine the path taken from the first node to the last, and they can be fixed (that is, after Node B, always visit node D) or conditional (evaluate a function to decide the next node to visit after node C).

LangGraph offers utilities to visualize these graphs and numerous features to debug their workings while in development. These graphs can then easily be deployed to serve production workloads at high scale.

If you followed the instructions in [Chapter 1](#), you'll already have LangGraph installed. If not, you can install it by running one of the following commands in your terminal:

Python

```
pip install langgraph
```

JavaScript

```
npm i @langchain/langgraph
```

To help get you familiar with using LangGraph, we'll create a simple chatbot using LangGraph, which is a great example of the LLM call architecture with a single use of an LLM. This chatbot will respond directly to user messages. Though simple, it does illustrate the core concepts of building with LangGraph.

Creating a StateGraph

Start by creating a `StateGraph`. We'll add a node to represent the LLM call:

Python

```
from typing import Annotated, TypedDict

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

class State(TypedDict):
    # Messages have the type "list". The `add_messages`
    # function in the annotation defines how this state should
    # be updated (in this case, it appends new messages to the
    # list, rather than replacing the previous messages)
    messages: Annotated[list, add_messages]

builder = StateGraph(State)
```

JavaScript

```

import {
  StateGraph,
  StateType,
  Annotation,
  messagesStateReducer,
  START, END
} from '@langchain/langgraph'

const State = {
  /**
   * The State defines three things:
   * 1. The structure of the graph's state (which "channels" are available
   * read/write)
   * 2. The default values for the state's channels
   * 3. The reducers for the state's channels. Reducers are functions that
   * determine how to apply updates to the state. Below, new messages are
   * appended to the messages array.
   */
  messages: Annotation({
    reducer: messagesStateReducer,
    default: () => []
  }),
}

const builder = new StateGraph(State)

```

NOTE

The first thing you do when you define a graph is define the state of the graph. The *state* consists of the shape, or schema, of the graph state, as well as reducer functions that specify how to apply updates to the state. In this example, the state is a dictionary with a single key: `messages`. The `messages` key is annotated with the `add_messages` reducer function, which tells LangGraph to append new messages to the existing list, rather than overwrite it. State keys without an annotation will be overwritten by each update, storing the most recent value. You can write your own reducer functions, which are simply functions that receive as arguments—argument 1 is the current state, and argument 2 is the next value being written to the state—and should return the next state, that is, the result of merging the current state with the new value. The simplest example is a function that appends the next value to a list and returns that list.

So now our graph knows two things:

- Every `node` we define will receive the current `State` as input and return a value that updates that state.
- `messages` will be *appended* to the current list, rather than directly overwritten. This is communicated via the prebuilt `add_messages` function in the `Annotated` syntax in the Python example or the reducer function for the JavaScript example.

Next, add the `chatbot` node. Nodes represent units of work. They are typically just functions:

Python

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI()

def chatbot(state: State):
    answer = model.invoke(state["messages"])
    return {"messages": [answer]}

# The first argument is the unique node name
# The second argument is the function or Runnable to run
builder.add_node("chatbot", chatbot)
```

JavaScript

```
import {ChatOpenAI} from '@langchain/openai'
import {
    AIMessage,
    SystemMessage,
    HumanMessage
} from '@langchain/core/messages';

const model = new ChatOpenAI()

async function chatbot(state) {
    const answer = await model.invoke(state.messages)
    return {"messages": answer}
}

builder = builder.addNode('chatbot', chatbot)
```

This node receives the current state, does one LLM call, and then returns an update to the state containing the new message produced by the LLM. The `add_messages` reducer appends this message to the messages already in the state.

And finally let's add the edges:

Python

```
builder.add_edge(START, 'chatbot')
builder.add_edge('chatbot', END)

graph = builder.compile()
```

```
builder = builder
  .addEdge(START, 'chatbot')
  .addEdge('chatbot', END)

let graph = builder.compile()
```

This does a few things:

- It tells the graph where to start its work each time you run it.
- This instructs the graph where it should exit (this is optional, as LangGraph will stop execution once there's no more nodes to run).
- It compiles the graph into a runnable object, with the familiar `invoke` and `stream` methods.

We can also draw a visual representation of the graph:

Python

```
graph.get_graph().draw_mermaid_png()
```

JavaScript

```
await graph.getGraph().drawMermaidPng()
```

The graph we just made looks like [Figure 4-6](#).

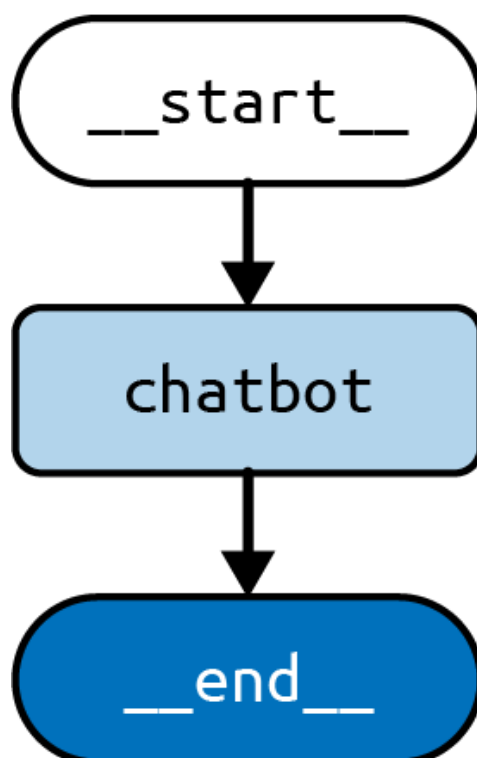


Figure 4-6. A simple chatbot

You can run it with the familiar `stream()` method you've seen in earlier chapters:

Python

```
input = {"messages": [HumanMessage('hi!)]}
for chunk in graph.stream(input):
    print(chunk)
```

JavaScript

```
const input = {messages: [new HumanMessage('hi!)]}
for await (const chunk of await graph.stream(input)) {
    console.log(chunk)
}
```

The output:

```
{ "chatbot": { "messages": [AIMessage("How can I help you?")] } }
```

Notice how the input to the graph was in the same shape as the `State` object we defined earlier; that is, we sent in a list of messages in the `messages` key of a dictionary. In addition, the `stream` function streams the full value of the state after each step of the graph.

Adding Memory to StateGraph

LangGraph has built-in persistence, which is used in the same way for the simplest graph to the most complex. Let's see what it looks like to apply it to this first architecture. We'll recompile our graph, now attaching a *checkpointer*, which is a storage adapter for LangGraph. LangGraph ships with a base class that any user can subclass to create an adapter for their favorite database; at the time of writing, LangGraph ships with several adapters maintained by LangChain:

- An in-memory adapter, which we'll use for our examples here
- A SQLite adapter, using the popular in-process database, appropriate for local apps and testing
- A Postgres adapter, optimized for the popular relational database and appropriate for large-scale applications.

Many developers have written adapters for other database systems, such as Redis or MySQL:

Python

```
from langgraph.checkpoint.memory import MemorySaver

graph = builder.compile(checkpointer=MemorySaver())
```

JavaScript

```
import {MemorySaver} from '@langchain/langgraph'

const graph = builder.compile({ checkpointer: new MemorySaver() })
```

This returns a runnable object with the same methods as the one used in the previous code block. But now, it stores the state at the end of each step, so every invocation after the first doesn't start from a blank slate. Any time the graph is called, it starts by using the checkpointer to fetch the most recent saved state, if any, and combines the new input with the previous state. And only then does it execute the first nodes.

Let's see the difference in action:

Python

```
thread1 = {"configurable": {"thread_id": "1"}}
result_1 = graph.invoke(
    { "messages": [HumanMessage("hi, my name is Jack!")] },
    thread1
)
// { "chatbot": { "messages": [AIMessage("How can I help you, Jack?")] } }

result_2 = graph.invoke(
    { "messages": [HumanMessage("what is my name?")] },
    thread1
)
// { "chatbot": { "messages": [AIMessage("Your name is Jack")] } }
```

JavaScript

```
const thread1 = {configurable: {thread_id: '1'}}
const result_1 = await graph.invoke(
    { "messages": [new HumanMessage("hi, my name is Jack!")] },
    thread1
)
// { "chatbot": { "messages": [AIMessage("How can I help you, Jack?")] } }

const result_2 = await graph.invoke(
    { "messages": [new HumanMessage("what is my name?")] },
```

```
thread1
)
// { "chatbot": { "messages": [AIMessage("Your name is Jack")] } }
```

Notice the object `thread1`, which identifies the current interaction as belonging to a particular history of interactions—which are called *threads* in LangGraph. Threads are created automatically when first used. Any string is a valid identifier for a thread (usually, Universally Unique Identifiers [UUIDs] are used). The existence of threads helps you achieve an important milestone in your LLM application; it can now be used by multiple users with independent conversations that are never mixed up.

As before, the `chatbot` node is first called with a single message (the one we just passed in) and returns another message, both of which are then saved in the state.

The second time we execute the graph on the same thread, the `chatbot` node is called with three messages, the two saved from the first execution, and the next question from the user. This is the essence of memory: the previous state is still there, which makes it possible, for instance, to answer questions about something said before (and do many more interesting things, of which we will see more later).

You can also inspect and update the state directly; let's see how:

Python

```
graph.get_state(thread1)
```

JavaScript

```
await graph.getState(thread1)
```

This returns the current state of this thread.

And you can update the state like this:

Python

```
graph.update_state(thread1, [HumanMessage('I like LLMs!)])
```

JavaScript

```
await graph.updateState(thread1, [new HumanMessage('I like LLMs!)])
```

This would add one more message to the list of messages in the state, to be used the next time you invoke the graph on this thread.

Modifying Chat History

In many cases, the chat history messages aren't in the best state or format to generate an accurate response from the model. To overcome this problem, we can modify the chat history in three main ways: trimming, filtering, and merging messages.

Trimming Messages

LLMs have limited *context windows*; in other words, there is a maximum number of tokens that LLMs can receive as a prompt. As such, the final prompt sent to the model shouldn't exceed that limit (particular to each mode), as models will either refuse an overly long prompt or truncate it. In addition, excessive prompt information can distract the model and lead to hallucination.

An effective solution to this problem is to limit the number of messages that are retrieved from chat history and appended to the prompt. In practice, we need only to load and store the most recent messages. Let's use an example chat history with some preloaded messages.

Fortunately, LangChain provides the built-in `trim_messages` helper that incorporates various strategies to meet these requirements. For example, the trimmer helper enables specifying how many tokens we want to keep or remove from chat history.

Here's an example that retrieves the last `max_tokens` in the list of messages by setting a strategy parameter to `"last"`:

Python

```
from langchain_core.messages import SystemMessage, trim_messages
from langchain_openai import ChatOpenAI

trimmer = trim_messages(
    max_tokens=65,
    strategy="last",
    token_counter=ChatOpenAI(model="gpt-4o"),
    include_system=True,
    allow_partial=False,
    start_on="human",
)

messages = [
```

```

        SystemMessage(content="you're a good assistant"),
        HumanMessage(content="hi! I'm bob"),
        AIMessage(content="hi!"),
        HumanMessage(content="I like vanilla ice cream"),
        AIMessage(content="nice"),
        HumanMessage(content="what's 2 + 2"),
        AIMessage(content="4"),
        HumanMessage(content="thanks"),
        AIMessage(content="no problem!"),
        HumanMessage(content="having fun?"),
        AIMessage(content="yes!"),
    ]

    trimmer.invoke(messages)

```

JavaScript

```

import {
    AIMessage,
    HumanMessage,
    SystemMessage,
    trimMessages,
} from "@langchain/core/messages";
import { ChatOpenAI } from "@langchain/openai";

const trimmer = trimMessages({
    maxTokens: 65,
    strategy: "last",
    tokenCounter: new ChatOpenAI({ modelName: "gpt-4o" }),
    includeSystem: true,
    allowPartial: false,
    startOn: "human",
});

const messages = [
    new SystemMessage("you're a good assistant"),
    new HumanMessage("hi! I'm bob"),
    new AIMessage("hi!"),
    new HumanMessage("I like vanilla ice cream"),
    new AIMessage("nice"),
    new HumanMessage("what's 2 + 2"),
    new AIMessage("4"),
    new HumanMessage("thanks"),
    new AIMessage("no problem!"),
    new HumanMessage("having fun?"),
    new AIMessage("yes!"),
]

const trimmed = await trimmer.invoke(messages);

```

The output:

```
[SystemMessage(content="you're a good assistant"),
 HumanMessage(content='what's 2 + 2'),
 AIMessage(content='4'),
 HumanMessage(content='thanks'),
 AIMessage(content='no problem!'),
 HumanMessage(content='having fun?'),
 AIMessage(content='yes!')] ]
```

Note the following:

- The parameter `strategy` controls whether to start from the beginning or the end of the list. Usually, you'll want to prioritize the most recent messages and cut older messages if they don't fit. That is, start from the end of the list. For this behavior, choose the value `last`. The other available option is `first`, which would prioritize the oldest messages and cut more recent messages if they don't fit.
- The `token_counter` is an LLM or chat model, which will be used to count tokens using the tokenizer appropriate to that model.
- We can add the parameter `include_system=True` to ensure that the trimmer keeps the system message.
- The parameter `allow_partial` determines whether to cut the last message's content to fit within the limit. In our example, we set this to `false`, which completely removes the message that would send the total over the limit.
- The parameter `start_on="human"` ensures that we never remove an `AIMessage` (that is, a response from the model) without also removing a corresponding `HumanMessage` (the question for that response).

Filtering Messages

As the list of chat history messages grows, a wider variety of types, sub-chains, and models may be utilized. LangChain's `filter_messages` helper makes it easier to filter the chat history messages by type, ID, or name.

Here's an example where we filter for human messages:

Python

```
from langchain_core.messages import (
    AIMessage,
    HumanMessage,
    SystemMessage,
    filter_messages,
)

messages = [
```



```

    SystemMessage("you are a good assistant", id="1"),
    HumanMessage("example input", id="2", name="example_user"),
    AIMessage("example output", id="3", name="example_assistant"),
    HumanMessage("real input", id="4", name="bob"),
    AIMessage("real output", id="5", name="alice"),
]

filter_messages(messages, include_types="human")

```

JavaScript

```

import {
  HumanMessage,
  SystemMessage,
  AIMessage,
  filterMessages,
} from "@langchain/core/messages";

const messages = [
  new SystemMessage({content: "you are a good assistant", id: "1"}),
  new HumanMessage({content: "example input", id: "2", name: "example_user"}),
  new AIMessage({content: "example output", id: "3", name: "example_assistant"}),
  new HumanMessage({content: "real input", id: "4", name: "bob"}),
  new AIMessage({content: "real output", id: "5", name: "alice"}),
];

filterMessages(messages, { includeTypes: ["human"] });

```

The output:

```

[HumanMessage(content='example input', name='example_user', id='2'),
 HumanMessage(content='real input', name='bob', id='4')]

```

Let's try another example where we filter to exclude users and IDs, and include message types:

Python

```

filter_messages(messages, exclude_names=["example_user", "example_assistant"],

"""
[SystemMessage(content='you are a good assistant', id='1'),
 HumanMessage(content='real input', name='bob', id='4'),
 AIMessage(content='real output', name='alice', id='5')]
"""

filter_messages(
    messages,
    include_types=[HumanMessage, AIMessage],

```

```

        exclude_ids=["3"]
    )

    """
    [HumanMessage(content='example input', name='example_user', id='2'),
     HumanMessage(content='real input', name='bob', id='4'),
     AIMessage(content='real output', name='alice', id='5')]
    """

```

JavaScript

```

filterMessages(
  messages,
  { excludeNames: ["example_user",
    "example_assistant"] }
);

/*
[SystemMessage(content='you are a good assistant', id='1'),
HumanMessage(content='real input', name='bob', id='4'),
AIMessage(content='real output', name='alice', id='5')]
*/

filterMessages(messages, { includeTypes: ["human", "ai"], excludeIds: ["3"]

/*
[HumanMessage(content='example input', name='example_user', id='2'),
  HumanMessage(content='real input', name='bob', id='4'),
  AIMessage(content='real output', name='alice', id='5')]
*/

```

The `filter_messages` helper can also be used imperatively or declaratively, making it easy to compose with other components in a chain:

Python

```

model = ChatOpenAI()

filter_ = filter_messages(exclude_names=["example_user", "example_assistant"]

chain = filter_ | model

```

JavaScript

```

const model = new ChatOpenAI()

const filter = filterMessages({
  excludeNames: ["example_user", "example_assistant"]
})

```

```
const chain = filter.pipe(model)
```

Merging Consecutive Messages

Certain models don't support inputs, including consecutive messages of the same type (for instance, Anthropic chat models). LangChain's `merge_message_runs` utility makes it easy to merge consecutive messages of the same type:

Python

```
from langchain_core.messages import (
    AIMessage,
    HumanMessage,
    SystemMessage,
    merge_message_runs,
)

messages = [
    SystemMessage("you're a good assistant."),
    SystemMessage("you always respond with a joke."),
    HumanMessage(
        [{"type": "text", "text": "i wonder why it's called langchain"}]
    ),
    HumanMessage("and who is harrison chasing anyway"),
    AIMessage(
        "'Well, I guess they thought \"WordRope\" and \"SentenceString\" just didn't have the same ring to it!'"
    ),
    AIMessage("""Why, he's probably chasing after the last cup of coffee in office!"""),
]

merge_message_runs(messages)
```

JavaScript

```
import {
  HumanMessage,
  SystemMessage,
  AIMessage,
  mergeMessageRuns,
} from "@langchain/core/messages";

const messages = [
  new SystemMessage("you're a good assistant."),
  new SystemMessage("you always respond with a joke."),
  new HumanMessage({
```

```

        content: [{ type: "text", text: "i wonder why it's called langchain" }
    ]),
    new HumanMessage("and who is harrison chasing anyway"),
    new AIMessage(
        `Well, I guess they thought "WordRope" and "SentenceString" just didn\
        have the same ring to it!`
    ),
    new AIMessage(
        "Why, he's probably chasing after the last cup of coffee in the office"
    ),
];

mergeMessageRuns(messages);

```

The output:

```

[SystemMessage(content="you're a good assistant.\nyou always respond with a
joke."),
HumanMessage(content=[{'type': 'text', 'text': "i wonder why it's called
langchain"}, 'and who is harrison chasing anyway']),
AIMessage(content='Well, I guess they thought "WordRope" and "SentenceStri
just didn\'t have the same ring to it!\nWhy, he\'s probably chasing aft
the last cup of coffee in the office!')]

```

Notice that if the contents of one of the messages to merge is a list of content blocks, then the merged message will have a list of content blocks. And if both messages to merge have string contents, then those are concatenated with a newline character.

The `merge_message_runs` helper can be used imperatively or declaratively, making it easy to compose with other components in a chain:

Python

```

model = ChatOpenAI()
merger = merge_message_runs()
chain = merger | model

```

JavaScript

```

const model = new ChatOpenAI()
const merger = mergeMessageRuns()
const chain = merger.pipe(model)

```

Summary

This chapter covered the fundamentals of building a simple memory system that enables your AI chatbot to remember its conversations with a user. We discussed how to automate the storage and updating of chat history using LangGraph to make this easier. We also discussed the importance of modifying chat history and explored various strategies to trim, filter, and summarize chat messages.

In [Chapter 5](#), you'll learn how to enable your AI chatbot to do more than just chat back: for instance, your new model will be able to make decisions, pick actions, and reflect on its past outputs.

Table of contents collapsed